Code Coverage Report (Line and Function Coverage)

Project: File Compression Tool (Basic ZIP Implementation): Team 14
Date: 05-Nov-2026
Code version: `main` @ `cae72ce` + Sprint-3 tests
Tool: GCC 13.3.0 `--coverage` (gcov) + gcovr 8.8
Detailed per-line report: `Documents/Test_Reports/coverage/index.html` (open it in a browser)

---

1. Result

Metric	Covered	Total	Coverage
Lines	340	379	**89.7%**
Functions	16	16	**100%**
Branches (see `Branch_Coverage_Report.md`)	401	487	82.3%

Per file

File	Module (SAD 3.4)	Lines	Line coverage	Functions
`src/frequency.cpp`	Compression Engine (REQ-2)	5 / 5	**100%**	1 / 1
`src/huffman.cpp`	Huffman Tree Module (REQ-3)	79 / 80	**98.8%**	7 / 7
`src/main.cpp`	CLI (REQ-1)	46 / 51	**90.2%**	4 / 4
`src/codec.cpp`	Compression / Decompression Engine, file writing (REQ-4 to REQ-7)	182 / 209	**87.1%**	2 / 2
`src/file_input.cpp`	File I/O (REQ-1)	28 / 34	**82.4%**	2 / 2
Total		**340 / 379**	**89.7%**	**16 / 16**

Every function in the program is executed by the tests. All lines that carry normal (non-error) behaviour are covered. The lines that are not covered are defensive error handlers, explained in Section 4.

---

2. Method

1. Every source file is compiled with `-O0 -g --coverage`.
2. The **complete test suite** is run against the instrumented build:
   - 36 unit tests (`run_tests`)

- 21 system validation test cases (`tests/system_validation.sh`)
- 17 lossless/corruption checks (`tests/lossless_check.sh`)

3. gcovr combines the results and reports on **`src/` only**. Test code is excluded so it doesn't inflate the numbers.

Options used: `--exclude-noncode-lines`, `--exclude-throw-branches`, `--exclude-unreachable-branches`. The last two affect branch numbers only; see the Branch Coverage Report.

Reproduce:

```
        pip install gcovr          # once
        bash tests/coverage.sh     # builds, runs all tests, writes
Documents/Test_Reports/coverage/
```

---

3. Improvement in Sprint 3

	Lines	Branches
Before Sprint-3 tests (unit + lossless + system tests)	306 / 379 = **80.7%**	346 / 487 = 71.0%
After adding `tests/test_robustness.cpp` (18 error-path tests) and TC-DEC-05	340 / 379 = **89.7%**	401 / 487 = 82.3%

The first coverage run showed that most damaged-archive and failed-I/O paths in `codec.cpp` had never been executed. The 18 new robustness tests (UT-ROB-01 to 18) exercise them and confirm each one exits cleanly without leaving an output file.

---

4. Lines not covered (39) and why

File : lines	Code	Why it is not covered	Risk
`codec.cpp` 74–76, 311–313	"Failed to write compressed/decompressed data"	Only happens if the disk fills up or the drive fails during the write. It can't be caused reliably on a normal test machine or in CI	Low: the temp file is deleted and an error is returned; the original data is never touched (D-14)
`codec.cpp` 83–85, 319–321	"Error renaming temp file"	Needs the operating system to refuse a rename inside a folder we could just write to (e.g. the file is locked by another program at that moment)	Low: same clean-up path as above
`codec.cpp` 326–327	Written file size ≠ original size after rename	Only possible on a disk error after a successful write	Low

File : lines	Code	Why it is not covered	Risk
`codec.cpp` 159–161	Exception while rebuilding the tree in decompress	**Unreachable:** the only exception `build_tree` throws is a frequency overflow, and the frequency-total check at lines 134–138 rejects that case first (UT-ROB-05)	None, defensive
`codec.cpp` 164–165	"Missing Huffman tree"	**Unreachable:** if the original size is not 0, at least one frequency is non-zero, so a tree always exists	None, defensive
`codec.cpp` 271–274	"Invalid Huffman bitstream" (a missing child)	**Unreachable:** a Huffman tree is a full binary tree, so every internal node has two children	None, defensive
`codec.cpp` 256–259	"Decoded data exceeds original size" in the **fast lookup path**	Needs a crafted payload whose extra symbols start exactly on a byte boundary. The same check in the bit-by-bit path (lines 284–288) **is** covered by UT-ROB-11	Low
`file_input.cpp` 25–26	File exists but cannot be opened (no read permission)	Needs a permission-denied file. Not testable as an administrator, and Windows handles permissions differently	Low
`file_input.cpp` 42–43, 47–48	Size cannot be determined / read fails half-way	Only on a device or stream error while reading	Low
`huffman.cpp` 149	Encoded bit count would exceed 2^{64}	Needs more than 2 exabytes of output	None
`main.cpp` 93–96, 98	`bad_alloc` / unexpected exception caught in `main`	Needs the program to run out of memory; kept as a safety net (SRS 6.2, D-14)	Low

Conclusion: none of the uncovered lines is part of normal compression or decompression. They are error handlers for hardware or OS failures, or defensive checks that the earlier validation already makes unreachable. All **reachable** error paths for user mistakes and damaged archives are covered.

---

5. Files in `Documents/Test_Reports/coverage/`

File	Content
`index.html`	Interactive report: summary, per file, per line (green = executed, red = not executed), with branch markers
`index.functions.html`	Function list with call counts

File	Content
`summary.txt`	Line coverage table (text)
`branches.txt`	Branch coverage table (text)
`coverage.json`	Machine-readable summary

Appendix: gcovr text output (lines)

```
------------------------------------------------------------------------------
                           GCC Code Coverage Report
Directory: .
------------------------------------------------------------------------------
File                                       Lines     Exec  Cover   Missing
------------------------------------------------------------------------------
src/codec.cpp                                209      182    87%   74-76,83-
85,159-161,164-165,256-259,271-274,311-313,319-321,326-327
src/file_input.cpp                            34       28    82%   25-26,42-
43,47-48
src/frequency.cpp                              5        5   100%
src/huffman.cpp                               80       79    98%   149
src/main.cpp                                  51       46    90%   93-96,98
------------------------------------------------------------------------------
TOTAL                                        379      340    89%
------------------------------------------------------------------------------
```